THE C LANGUAGE · PRACTICE SERIES

Pointers in C



Ten exercises, from your first * to a linked list — each one ships with the notions you need to solve it.

```
int x = 42;  int *p = &x;
```

10 exercises | beginner → advanced | solutions in the companion deck

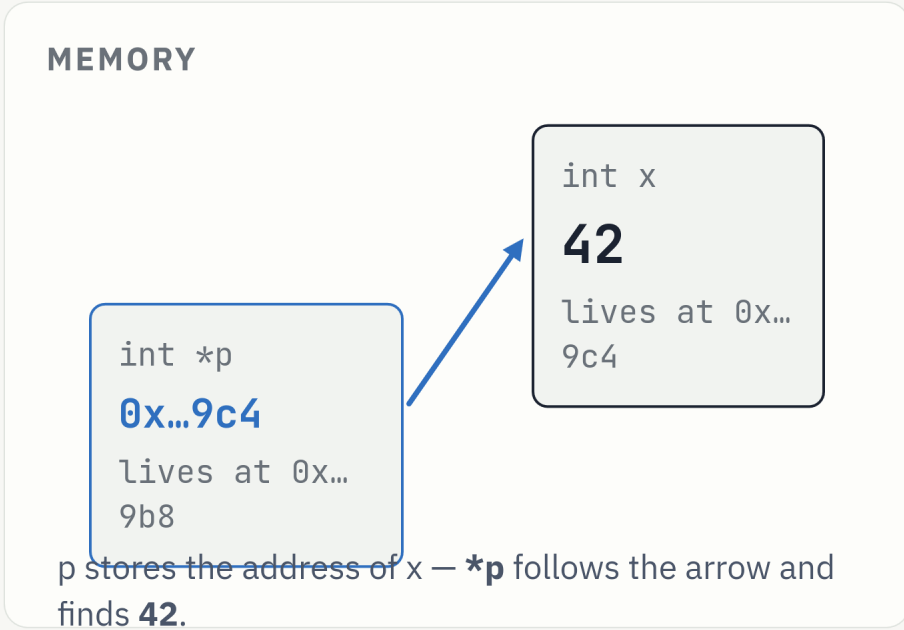
BEFORE YOU START

What a pointer actually is

- Memory is one long row of numbered cells — a cell's number is its **address**.
- **int x = 42;** reserves a few cells (4 bytes here) and names them **x**.
- A **pointer** is a variable whose value is an address: **int *p = &x;** stores "where x lives".
- The type matters: an **int *** knows it steps over ints — that becomes pointer arithmetic in exercise 4.

`&x → the address of x` `*p → the value at p`

Two operators, two directions: **&** takes an address, ***** follows one.



THE PROGRESSION

Ten exercises, three stages

BEGINNER · 01–03

01 · Your first pointer
`&x, *p`

02 · Pointers that point nowhere
`NULL`

03 · Functions that write back
`swap(&u, &v)`

INTERMEDIATE · 04–07

04 · Walking an array
`*(a + i), p++`

05 · Strings, one char at a time
`'\0'`

06 · A pointer to a pointer
`int **`

07 · The heap
`malloc, free`

ADVANCED · 08–10

08 · A dynamic matrix
`int **m`

09 · Pointers to functions
`int (*f)(int, int)`

10 · Capstone: a linked list
`struct node **`

Each exercise slide lists the notions you need. Try on your own first — the companion deck has every solution, commented.

EXERCISE 01 · BEGINNER

Your first pointer

YOUR TASK

1. Declare **int x = 42;** then a pointer **p** holding the address of **x**.
2. Print **x**, the address of **x**, the value of **p**, and the value **p** points to.
3. Without naming **x**, set the pointed value to **100** — then print **x** again.

EXPECTED OUTPUT

```
x  = 42
&x = 0x7ffd…9c4  (yours will differ)
p  = 0x7ffd…9c4
*p = 42
x  = 100
```

WHAT YOU NEED TO KNOW

`int *p;`

"p is a pointer to int" — it stores an address, not an int.

`&x`

Address-of: where x lives in memory.

`*p`

Dereference: the object at that address. You can read it — or assign into it.

`printf("%p", (void *)p)`

How you print an address: %p expects a void pointer.

EXERCISE 02 · BEGINNER

Pointers that point nowhere

YOUR TASK

1. Write **void safe_print(const int *p)** — print the pointed value, or **(null pointer)** when there is nothing to read.
2. In **main**, call it twice: once with **&x**, once with **NULL**.
3. In a comment, explain why an *uninitialized* pointer would slip past your guard.

EXPECTED OUTPUT

```
value: 7
(null pointer)
```

WHAT YOU NEED TO KNOW

`NULL`

The official "points nowhere" value — the only safe default for a pointer.

`*p on NULL or garbage`

Undefined behavior: often a crash, sometimes silent corruption — worse.

`if (p ≠ NULL)`

Guard first, dereference second. if (p) is the same test.

`const int *p`

A promise: this function only reads through p, never writes.

EXERCISE 03 · BEGINNER

Functions that write back

YOUR TASK

1. Write **void swap(int *a, int *b)** exchanging two ints.
2. In **main**: **int u = 3, v = 7;** — after your call, u is 7 and v is 3. Print before and after.
3. In a comment: why can **void swap(int a, int b)** never work?

EXPECTED OUTPUT

```
before  u=3 v=7
after   u=7 v=3
```

WHAT YOU NEED TO KNOW

pass by value

C always copies arguments — the callee never sees your variables...

swap(&u, &v)

...unless you hand it their addresses instead.

***a, *b**

Inside the function these are u and v. You need a temp for the three-step exchange.

the "out parameter" pattern

The same idea behind scanf("%d", &n) — keep it in your pocket.

EXERCISE 04 · INTERMEDIATE

Walking an array

YOUR TASK

1. Implement **int sum(const int *a, size_t n) — no square brackets anywhere**, pointer arithmetic only.
2. Walk with a pointer: start at the first element, stop one past the last.
3. In **main**: **int t[] = {2, 4, 6, 8, 10};** — compute n with **sizeof t / sizeof *t**.

EXPECTED OUTPUT

```
sum = 30
```

WHAT YOU NEED TO KNOW

decay

Passed to a function, an array arrives as a pointer to its first element.

a[i] ≡ *(a + i)

Brackets are pointer arithmetic in disguise.

p++

Advances by one element — the type scales the step, never one byte.

p < a + n

One-past-the-end: legal to compare against, never to dereference.

EXERCISE 05 · INTERMEDIATE

Strings, one char at a time

YOUR TASK

1. Implement **size_t my_strlen(const char *s)** and **char *my_strcpy(char *dst, const char *src)** — no <string.h>, no brackets.
2. **my_strcpy** returns **dst**, like the real one.
3. In **main**: copy **"pointers are easy"** into **char buf[32]**; print the copy and its length.

EXPECTED OUTPUT

```
pointers are easy (17)
```

WHAT YOU NEED TO KNOW

`'\0'`

A C string is just chars followed by a zero byte. That sentinel is the whole contract.

`while (*s) s++;`

The canonical walk to the end of a string.

`p - s`

Two pointers into the same array subtract to an element count.

`literals are read-only`

Write into your own buffer — and make it big enough. C won't check for you.

EXERCISE 06 · INTERMEDIATE

A pointer to a pointer

YOUR TASK

1. Write **void point_to_max(int **sel, int *a, int *b)** — after the call, ***sel** points to the larger of the two ints.
2. In **main**: **int x = 3, y = 9; int *m = NULL;** — call it, then execute ***m = 0;**
3. Predict before running: which of x, y just changed? Verify with a print.

EXPECTED OUTPUT

```
x=3 y=0
```

WHAT YOU NEED TO KNOW

`int **sel`

A pointer to a pointer: *sel is an int *, **sel is the int itself.

`exercise 3, one floor up`

To modify the caller's pointer, take the pointer's address.

`*sel = a;`

Rewires the caller's m: an address is copied, no int moves.

`read right to left`

"sel is a pointer to a pointer to int."

EXERCISE 07 · INTERMEDIATE

The heap: malloc and free

YOUR TASK

1. Allocate an array of **n = 8** ints on the heap; fill **a[i] = i*i**; print all values. (Brackets are legal again.)
2. If allocation fails: message on **stderr**, exit non-zero.
3. **free** the array, then set the pointer to **NULL**. Run under **valgrind**: zero leaks.

EXPECTED OUTPUT

```
0 1 4 9 16 25 36 49
```

WHAT YOU NEED TO KNOW

`malloc(n * sizeof *a)`

Heap memory that outlives the function — and NULL on failure: always check.

`sizeof *a`

"The size of what a points to" — survives a change of type.

`free(a)`

Exactly once per malloc; afterwards a dangles until you re-point it.

`the three heap sins`

The leak, the double free, the use-after-free. valgrind sees them all.

EXERCISE 08 · ADVANCED

A dynamic matrix

YOUR TASK

1. **int **make_matrix(size_t rows, size_t cols)** — allocate an array of row pointers, then each row; fill **m[r][c] = r*cols + c**.
2. If a row allocation fails: free everything already allocated, return NULL.
3. **void free_matrix(int **m, size_t rows)** — release it all, in the right order. valgrind: clean.

EXPECTED OUTPUT — 3 × 4

```
 0  1  2  3
 4  5  6  7
 8  9 10 11
```

WHAT YOU NEED TO KNOW

`int **m`

Here: a malloc'd array of int *, one per row — 1 + rows allocations in total.

`m[r][c]`

No 2-D magic: it is *(*(m + r) + c) — two hops.

`free in reverse`

Every row first, the row-pointer array last.

`fail-path hygiene`

Unwind what already exists — no path through the function may leak.

EXERCISE 09 · ADVANCED

Pointers to functions

YOUR TASK

1. Write **add**, **sub**, **mul** — each takes two ints, returns an int.
2. Store the three in an **array of function pointers**, plus a parallel array of names; one loop prints every result for (6, 7).
3. Bonus: sort an int array with **qsort** and your own comparator.

EXPECTED OUTPUT

```
add(6, 7) = 13
sub(6, 7) = -1
mul(6, 7) = 42
```

WHAT YOU NEED TO KNOW

`functions have addresses`

A function name decays to a pointer to its code.

`int (*f)(int, int)`

The parentheses are vital — without them you declare a function returning int *.

`f(6, 7)`

Calling is plain — no star needed; (*f)(6, 7) is the same call.

`dispatch tables`

int (*ops[])(int, int) — the mechanism behind qsort, callbacks, plugins.

EXERCISE 10 · ADVANCED

Capstone: a linked list

YOUR TASK

1. Define **struct node { int value; struct node *next; };**
2. Implement **push_front(struct node **head, int v)**, **print_list(const struct node *h)** and **free_list(struct node **head)** — free_list leaves head at NULL.
3. In **main**: start from NULL; push 1, 2, 3; print; free. valgrind: zero leaks.

EXPECTED OUTPUT

```
3 → 2 → 1 → NULL
```

WHAT YOU NEED TO KNOW

`self-reference`

Each node stores a pointer to the next; NULL marks the end.

`p→next`

Shorthand for (*p).next — dereference plus member access in one arrow.

`struct node **head`

push_front must rewrite the caller's head — exercise 6 at work.

`the full toolkit`

One malloc per node, one free per node, walk with p = p->next.

BEFORE YOU DIVE IN

How to work through the deck

WORKFLOW

- One file per exercise: **ex01.c**, **ex02.c**, ...
- Compile with **gcc -std=c17 -Wall -Wextra -g** — with pointers, a warning is a bug.
- From exercise 7 on, finish with **valgrind ./ex07** — "zero leaks" is part of done.
- Stuck for 20 minutes? Read only the "why it works" column of the solution, then retry alone.

GOLDEN RULES

- ✓ Initialize every pointer — to a real address or NULL.
- ✓ Check before you dereference.
- ✓ Every malloc gets exactly one free.
- ✓ Never return the address of a local variable.
- ✓ When lost, draw the boxes and arrows on paper.

Companion deck: **C Pointers — 10 Solutions** · every exercise solved and commented, with the why behind each line.